AWS Plugin for VPC Resource Monitoring

Configure the Panorama AWS Plugin to dynamically monitor EC2 instances, map metadata to IP tags, and enforce security policy through Dynamic Address Groups

GUIDE APPROACH

This guide configures the **AWS Plugin for Panorama** to monitor EC2 instance metadata across AWS VPCs and push IP-to-tag mappings to managed firewalls. Firewalls consume these tags through **Dynamic Address Groups** referenced in security policy — enabling policy that adapts automatically as EC2 instances are created, terminated, or re-tagged.

Prerequisite: Panorama must already be deployed, licensed, and managing VM-Series firewalls organized into Device Groups. If Panorama is not yet deployed, complete the Panorama Deployment Guide first.

Architecture Overview

The AWS plugin runs as a daemon on Panorama. It polls AWS VPCs at a configurable interval (default 60 seconds), retrieves EC2 instance metadata, creates IP-to-tag mappings from predefined and user-defined attributes, and pushes those tags to firewalls in specified Device Groups. Firewalls use the tags in Dynamic Address Groups referenced by security policy rules — creating policy that automatically adapts as the cloud environment changes.

Data Flow Pipeline

Tag information flows through five stages from AWS to the firewall dataplane:

STAGE	COMPONENT	ACTION
1. Poll	Tag Retrieval Daemon	Queries AWS APIs for EC2/ELB metadata across configured VPCs
2. Process	Tag Processing Daemon	Filters invalid tags, computes diff from previous poll
3. Push	Panorama configd	Sends incremental tag updates to firewalls in Notify Group device groups
4. Register	Firewall userid	Registers IP-to-tag mappings, updates Dynamic Address Groups
5. Enforce	Firewall dataplane	Security policy rules using DAGs automatically match updated IPs

ℹ️ **PLUGIN CAPACITY**

The AWS plugin version 3.0.1 or later supports monitoring EC2 instances for up to 1,000 AWS VPCs across AWS Commercial, AWS GovCloud, and AWS China regions. Panorama cannot be deployed on AWS China; provide AWS credentials for China regions instead of using an instance profile.

ℹ️ **PULL MODEL — NOT PUSH**

The plugin operates on a pull model — Panorama actively polls AWS. AWS never pushes data to Panorama or the firewalls. Only metadata is retrieved; no traffic or logs flow through this channel.

✅ **VERIFICATION**

Confirm understanding of the five-stage pipeline before proceeding. Each remaining section of this guide configures one or more stages in this flow.

Phase 1: Prerequisites

Gather requirements and make design decisions before configuring IAM or installing the plugin.

Step 1.1: Verify Panorama Minimum Requirements

Panorama resource requirements scale with the number of monitored VPCs and the tag density per IP address.

MEMORY	CPUS	MONITORED VPCS	TAG CAPACITY
16 GB	4	1–100	10,000 IPs × 13 tags each
32 GB	8	100–500	5,000 IPs × 25 tags each
64 GB	16	500–1,000	5,000 IPs × 25 tags each

Additional requirements:

- PAN-OS `10.0.5` or later on Panorama
- AWS Plugin version `3.0.1` or later
- Active support license on Panorama
- Device management license for managed firewalls
- Valid support license on managed firewalls

✅ **VERIFICATION**

Log in to Panorama. Navigate to **Panorama > Dashboard** and confirm the PAN-OS version is `10.0.5` or later. Check **Panorama > Support** to confirm the support license is active.

Step 1.2: Gather AWS Information

Collect the following information from the AWS Console before starting IAM or plugin configuration:

- **VPC IDs** for each VPC to monitor (AWS Console > VPC Dashboard > Your VPCs)
- **AWS Account IDs** for each account containing monitored VPCs
- **AWS Regions** where monitored VPCs are located
- **Panorama deployment location** — on AWS (EC2) or on-premises — this determines the authentication method
- **EC2 instance tags** — tag EC2 instances with the key-value pairs to use in security policy before enabling monitoring
- **Duplicate IP check** — identify any overlapping IP addresses across monitored VPCs

> ⚠️ **IMPORTANT: DUPLICATE IPS ACROSS VPCS**
>
> Duplicate IP addresses across monitored VPCs cause metadata to be appended together or swapped between instances. This may produce unexpected results in policy enforcement. Duplicate IPs are logged to `plugin_aws_ret.log` on Panorama.

> ✅ **VERIFICATION**
>
> All VPC IDs, Account IDs, and region names are recorded and accessible. EC2 instances in those VPCs have the tag key-value pairs intended for security policy.

Step 1.3: Choose an Authentication Method

The authentication method depends on where Panorama is deployed and whether monitored VPCs are in the same or different AWS accounts.

OPTION	WHEN TO USE	CREDENTIALS ON PANORAMA
1. IAM Credentials	Panorama not on AWS, single account	Access Key ID + Secret Access Key
2. IAM Credentials + Assume Role	Panorama not on AWS, cross-account	Access Key + Secret Key + Role ARN
3. Instance Profile	Panorama on AWS, same account	None (automatic via instance profile)
4. Instance Profile + Assume Role	Panorama on AWS, cross-account	Role ARN only

✅ **VERIFICATION**

If Panorama is deployed on AWS, use Instance Profile (Option 3 or 4) — it eliminates the need to store AWS credentials on Panorama and is the most secure approach. Record the chosen option number; the IAM setup phase references it directly.

Step 1.4: Confirm Firewall Readiness

The plugin pushes tags to firewalls through Panorama Device Groups. Confirm the following before proceeding:

- Firewalls are added as managed devices on Panorama (**Panorama > Managed Devices > Summary**)
- Firewalls are organized into Device Groups (**Panorama > Device Groups**)
- Target Device Groups are **not** already attached to another Notify Group's parent Device Group

> ✅ **VERIFICATION**
>
> Navigate to **Panorama > Managed Devices > Summary**. All target firewalls show `Connected` status. Navigate to **Panorama > Device Groups** and confirm the intended Device Groups exist with firewalls assigned.

Phase 2: IAM Roles and Permissions

Create the IAM resources required for Panorama to authenticate with AWS and retrieve EC2 metadata. Complete only the option matching the authentication method chosen in Step 1.3.

Step 2.1: Define the Required IAM Permissions

All authentication options use the same minimum permission set. This policy grants read-only access to EC2 instance metadata and ELB attributes — no write permissions.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PanoramaVMMonitoring",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeNetworkInterfaces",
        "ec2:DescribeVpcs",
        "ec2:DescribeVpcEndpoints",
        "ec2:DescribeSubnets",
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeLoadBalancerAttributes",
        "elasticloadbalancing:DescribeTags"
      ],
      "Resource": "*"
    }
  ]
}
```

ℹ️ **MINIMUM PERMISSION SET**

The EC2 Describe actions retrieve instance metadata (IPs, tags, VPC, subnet, security group). The ELB Describe actions retrieve load balancer attributes for tagging. All actions are read-only — the plugin never modifies AWS resources.

✅ **VERIFICATION**

Save this JSON as a local file or keep it accessible. It is referenced in every authentication option below.

Step 2.2: Option 1 — IAM Credentials (Single Account)

Use this option when Panorama is **not** deployed on AWS and all monitored VPCs are in a single AWS account.

1. Open AWS Console > **IAM** > **Policies** > **Create Policy**.
2. Select the **JSON** tab and paste the permission policy from Step 2.1.
3. Click **Next**, name the policy `PanoramaVMMonitoring`, and click **Create policy**.
4. Navigate to **IAM** > **Users** > **Create user**.
5. Enter `panorama-monitoring` as the user name. Do not enable console access.
6. On the Permissions page, select **Attach policies directly** and attach `PanoramaVMMonitoring`.
7. Click **Create user**.
8. Open the new user > **Security credentials** > **Create access key**.
9. Select **Third-party service** as the use case, acknowledge the warning, and click **Create access key**.
10. Copy the **Access Key ID** and **Secret Access Key**.

> ⚠️ **IMPORTANT: STORE CREDENTIALS SECURELY**
>
> The Secret Access Key is shown only once during creation. Store it in a secure vault. If lost, delete the access key and create a new one.

> ✅ **VERIFICATION**
>
> The IAM user `panorama-monitoring` appears under **IAM > Users** with the `PanoramaVMMonitoring` policy attached. An active access key exists under **Security credentials**.

Step 2.3: Option 2 — IAM Credentials + Assume Role (Cross-Account)

Use this option when Panorama is **not** deployed on AWS and monitored VPCs span multiple AWS accounts. This requires configuration in both accounts.

On Account 1 (monitored account — where EC2 instances are)

1. Navigate to **IAM** > **Roles** > **Create role**.
2. Select **Another AWS account** as the trusted entity.
3. Enter Account 2's Account ID (the account where Panorama credentials are stored).
4. Attach the `PanoramaVMMonitoring` policy from Step 2.1.
5. Name the role (e.g., `PanoramaMonitoringRole`) and click **Create role**.
6. Copy the **Role ARN** (e.g., `arn:aws:iam::111111111111:role/PanoramaMonitoringRole`).

The trust policy on this role should look like:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT-2-ID:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

On Account 2 (Panorama's credentials account)

1. Create an IAM user (e.g., `panorama-monitoring`).
2. Attach an inline or managed policy allowing `sts:AssumeRole` :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::ACCOUNT-1-ID:role/PanoramaMonitoringRole"
    }
  ]
}
```

1. Create access keys for this user and save the Access Key ID and Secret Access Key.
2. On Panorama, enter the Access Key ID and Secret Key from Account 2, and provide the Role ARN from Account 1 in the Monitoring Definition.

ℹ️ **MULTIPLE MONITORED ACCOUNTS**

Repeat the Account 1 steps for each additional monitored account. Each account needs its own IAM role with Account 2 as a trusted entity. Create a separate Monitoring Definition on Panorama for each account's Role ARN.

✅ **VERIFICATION**

On Account 1: the role `PanoramaMonitoringRole` exists with the VM Monitoring policy and Account 2 as a trusted entity. On Account 2: the IAM user has an active access key and the `sts:AssumeRole` policy referencing Account 1's role.

Step 2.4: Option 3 — Instance Profile (Same Account)

Use this option when Panorama runs as an EC2 instance in the **same AWS account** as the monitored VPCs. No credentials are stored on Panorama.

1. Navigate to **IAM** > **Roles** > **Create role**.
2. Select **AWS service** as the trusted entity and choose **EC2**.
3. Attach the `PanoramaVMMonitoring` policy from Step 2.1.
4. Name the role (e.g., `PanoramaInstanceProfile`) and click **Create role**.
5. Navigate to **EC2** > **Instances**, select the Panorama instance.
6. Click **Actions** > **Security** > **Modify IAM role**.
7. Select the `PanoramaInstanceProfile` role and click **Update IAM role**.

ℹ️ **INSTANCE PROFILE AUTO-CREATION**

When creating an IAM role using the AWS Console, AWS automatically creates an instance profile with the same name. When Panorama launches with this role, the instance profile is automatically associated.

✅ **VERIFICATION**

In the EC2 Console, select the Panorama instance and check the **Security** tab. The **IAM Role** field shows `PanoramaInstanceProfile`. On Panorama, select `Instance Profile` as the Account Type — no credentials are needed.

Step 2.5: Option 4 — Instance Profile + Assume Role (Cross-Account)

Use this option when Panorama runs as an EC2 instance on AWS but monitored VPCs are in **different AWS accounts**.

On Account 1 (monitored account — where EC2 instances are)

1. Navigate to **IAM** > **Roles** > **Create role**.
2. Select **Another AWS account** as the trusted entity.
3. Enter Account 2's (Panorama's) Account ID.
4. Attach the `PanoramaVMMonitoring` policy from Step 2.1.
5. Name the role and click **Create role**.
6. Copy the **Role ARN**.

On Account 2 (Panorama's account)

1. Create an IAM role with **EC2** as the trusted entity.
2. Attach an `sts:AssumeRole` policy referencing Account 1's Role ARN.
3. Attach the instance profile to the Panorama EC2 instance (same process as Step 2.4, steps 5–7).

For each additional monitored account, create a role on that account with the VM Monitoring policy and add Account 2 as a trusted entity.

On Panorama:

- Select `Instance Profile` as Account Type
- Enter the Role ARN from Account 1 in the Monitoring Definition

✅ **VERIFICATION**

The Panorama EC2 instance has an instance profile attached (visible in the EC2 Console > Security tab). On Account 1, the monitoring role lists Account 2 as a trusted entity. The Role ARN from Account 1 is recorded for use in the Monitoring Definition.

Phase 3: Install the AWS Plugin

Download and install the AWS plugin on Panorama. This is a one-time operation per Panorama instance (or HA pair).

Step 3.1: Download and Install

1. Log in to the Panorama web interface.
2. Navigate to **Panorama > Plugins**.
3. Click **Check Now** to refresh the plugin list from the Palo Alto Networks update server.
4. Locate `aws` in the list. Download the latest `3.x` version.
5. After the download completes, click **Install**.
6. Wait for Panorama to refresh. The AWS plugin appears on the Plugins tab.

⚠️ **IMPORTANT: MAINTENANCE WINDOW**

Install or uninstall plugins during a planned maintenance window. Installing (or uninstalling) a plugin requires a Panorama reboot if another cloud plugin is already installed.

⚠️ **IMPORTANT: NO DOWNGRADE FROM V2.0+**

After installing the AWS plugin v2.0 or later, downgrading to v1.0 is not supported.

✅ **VERIFICATION**

The plugin appears in the **Panorama > Plugins** list with status `Installed` and the expected version number.

Step 3.2: Verify Installation

1. Navigate to **Panorama > Dashboard**. Check the General Information widget for the plugin version.
2. Navigate to **Panorama > Plugins > AWS**. Confirm the plugin management interface loads.

> ℹ️ **PANORAMA HA**
>
> If Panorama is configured for HA, install the same plugin version on both peers and attach the same instance profile (if applicable) to both Panorama EC2 instances.

> ✅ **VERIFICATION**
>
> The **Panorama > Plugins > AWS** navigation item is visible and loads the Setup, Monitoring Definition, and Notify Group tabs.

Step 3.3: Commit (HA Only)

Synchronize the plugin installation to the passive Panorama peer.

1. Navigate to **Commit > Commit to Panorama**.
2. Click **Commit**.

> ℹ️ **UNCONFIGURED PLUGINS BLOCKING TAG PUSH**
>
> If multiple Panorama plugins are installed and one is not configured, Panorama will not forward IP-tag information to unconfigured plugins. Either uninstall unused plugins, or run the following command to unblock:

```
request plugins dau plugin-name <plugin-name> unblock-device-push yes
```

> ✅ **VERIFICATION**
>
> The commit completes without errors. On the passive Panorama peer, **Panorama > Plugins > AWS** shows the same installed version.

Phase 4: Configure the AWS Plugin

Enable monitoring, create Notify Groups, add IAM credentials, and define which VPCs to monitor. Complete these steps in order — each depends on the previous.

Step 4.1: Enable Monitoring

1. Navigate to **Panorama > Plugins > AWS > Setup > General**.
2. Check **Enable Monitoring**.
3. Set **Monitoring Interval** (default: `60` seconds).

> ℹ️ **MONITORING INTERVAL TRADE-OFF**
>
> The monitoring interval determines how frequently Panorama polls AWS for metadata changes. Lower values increase API call volume to AWS. 60 seconds is recommended for most deployments.

> ✅ **VERIFICATION**
>
> The **Enable Monitoring** checkbox is checked and the interval value is displayed in the General settings panel.

Step 4.2: Create Notify Group

A Notify Group defines which Device Groups (and their firewalls) receive IP-to-tag mappings from the plugin.

1. Navigate to **Panorama > Plugins > AWS > Setup > Notify Groups**.
2. Click **Add**.
3. Enter a **Name** (e.g., `NG-Production`).
4. Select **Device Groups** — choose all Device Groups whose firewalls should receive tags.
5. Select tags: **Select All 32 Tags** (default) or **Custom Tags** to select specific predefined and user-defined tags.
6. Click **OK**.

⚠️ **IMPORTANT: DEVICE GROUP ASSIGNMENT RULES**

Think through Device Groups carefully:

- A Monitoring Definition can include only one Notify Group — select all relevant Device Groups within it
- To unregister tags from a firewall in a Notify Group, delete the Monitoring Definition
- To register tags to all virtual systems on a multi-vsys firewall, add each virtual system to a separate device group and assign all to the Notify Group

ℹ️ **CUSTOM TAGS FOR LARGE DEPLOYMENTS**

When monitoring a large number of EC2 instances, use Custom Tags to select only the tags needed for security policy. This reduces memory consumption on both Panorama and the firewalls.

✅ **VERIFICATION**

The Notify Group appears in the list with the correct Device Groups and tag selection displayed.

Step 4.3: Add IAM Role

Register the IAM credentials or instance profile that the plugin uses to authenticate with AWS.

1. Navigate to **Panorama > Plugins > AWS > Setup > IAM Role**.
2. Click **Add**.
3. Enter a **Name** and optional **Description**.
4. Select **Account Type**:
   - **Instance Profile** — if Panorama runs on AWS with an attached instance profile (Options 3 or 4 from Step 1.3)
   - **AWS Account Credentials** — if Panorama is not on AWS or explicit credentials are required (Options 1 or 2)
5. (For AWS Account Credentials) Enter the **Access Key ID** and **Secret Access Key**. Confirm the secret key.
6. Click **OK**.

> **AWS GOVCLOUD REGION SETTING**
> For AWS GovCloud, set the AWS region using the CLI before adding the IAM role:

```
request plugins aws set-aws-region region <aws-govcloud-region>
```

After adding, the IAM Roles list shows a validation status icon:

COLOR	STATUS	MEANING
Green	Valid	Credentials verified for monitoring
Orange	Valid for Monitoring	Credentials work for monitoring but not deployments (expected for monitoring-only roles)
Red	Invalid	Credentials failed validation — check Access Key and permissions

✅ **VERIFICATION**

The IAM Role entry appears in the list with a **Green** or **Orange** validation status. Red status indicates a credential or permission problem — revisit Phase 2.

Step 4.4: Create Monitoring Definition

A Monitoring Definition ties together an IAM role, a set of AWS regions and VPCs, and a Notify Group. Create one Monitoring Definition per AWS account being monitored.

1. Navigate to **Panorama > Plugins > AWS > Monitoring Definition > General**.
2. Click **Add**.
3. Enter a **Name** and optional **Description**.
4. Select the **IAM Role** created in Step 4.3.
5. Select **AWS Regions**:
    - **All** — monitor all AWS regions
    - **Select** — choose specific regions from the Member search bar
6. (Optional) Enter a **Role ARN** if using Assume Role for cross-account access (Options 2 or 4).
7. Select the **Notify Group** created in Step 4.2.
8. Check **Enable**.
9. Click **OK**.
10. Switch to the **VPC IDs** tab.
11. Click **Add** and enter VPC IDs from the AWS VPC Dashboard.
12. Click **OK**.

> ℹ️ **GOVCLOUD MONITORING DEFINITIONS**
>
> For AWS GovCloud, configure the AWS region under the monitoring definition using CLI:

```
set plugins aws monitoring-definition <name> aws-regions <aws-govcloud-region>
```

> ✅ **VERIFICATION**
>
> The Monitoring Definition appears in the list with the correct IAM Role, region selection, Notify Group, and VPC IDs. The **Enable** checkbox is checked.

Step 4.5: Commit and Validate

Commit the plugin configuration to activate monitoring and validate connectivity to AWS.

1. Navigate to **Commit > Commit to Panorama**. Click **Commit**.
2. After the commit completes, return to **Panorama > Plugins > AWS > Monitoring Definition**.
3. Verify the **Status** column shows `Success`.
4. Click **Validate** to confirm Panorama can authenticate and communicate with the configured VPCs.

If status shows failure:

- Verify the VPC ID is correct (check for typos)
- Verify the IAM credentials or role have the required permissions from Step 2.1
- Verify Panorama has outbound internet connectivity to AWS API endpoints
- Check `plugin_aws_ret.log` for detailed error messages

> ✅ **VERIFICATION**
> After commit, the plugin begins polling AWS immediately. Within one monitoring interval (default 60 seconds), the Status column changes to `Success` with IP-to-tag mappings populated. The Validate button confirms connectivity.

Phase 5: Create Dynamic Address Groups

Dynamic Address Groups (DAGs) use tag match criteria to automatically include IPs that carry matching tags. Create DAGs on Panorama, then reference them in security policy rules.

Step 5.1: Create a Dynamic Address Group

1. Navigate to **Objects > Address Groups** (in the Device Group selected in the Notify Group).
2. Click **Add**.
3. Enter a **Name** (e.g., `DAG-WebServers`).
4. Set **Type** to `Dynamic`.
5. In the **Match** field, enter the tag match criteria.

Tag match criteria examples:

```
'aws.ec2.tag.Environment.Production'

'aws.ec2.vpc-id.vpc-0abc123def456'

'aws.ec2.subnet-id.subnet-0abc123' and 'aws.ec2.tag.Role.WebServer'

'aws.ec2.tag.Environment.Production' or 'aws.ec2.tag.Environment.Staging'
```

1. Click **OK**.

> **MATCH CRITERIA SYNTAX**
>
> Wrap each tag value in single quotes. Use `and` / `or` operators for boolean logic. Boolean operations require PAN-OS 9.1.6 or later.

> **DAG BEHAVIOR DIFFERENCE: PANORAMA VS. FIREWALL**
>
> DAGs on Panorama always show associated IPs. On firewalls, DAGs only show IPs when the DAG is referenced in an active security policy rule.

> ✅ **VERIFICATION**
>
> The DAG appears in **Objects > Address Groups** with Type `Dynamic` and the match criteria displayed in the Match column.

Step 5.2: Verify DAG Population

Confirm that the DAG resolves to the expected IP addresses on Panorama.

1. Navigate to **Objects > Address Groups**.
2. Find the DAG and click **more** in the Addresses column.
3. Verify the expected IPs appear in the list.

If no IPs appear:

- Confirm the tag name in the match criteria matches exactly — run `show object registered-ip all` on a firewall in the Notify Group to see actual tag names
- Confirm the Device Group is in the Notify Group
- Confirm the Monitoring Definition status is `Success`
- Wait at least one monitoring interval after the initial commit

> ✅ **VERIFICATION**
>
> Clicking **more** on the DAG shows the private IP addresses of EC2 instances matching the tag criteria. The IP count matches the number of running instances with the specified tags in AWS.

Phase 6: Create Security Policy Rules

Reference Dynamic Address Groups in security policy rules. When DAG membership changes (new EC2 instances, tag changes), policy enforcement updates automatically — no commit required for tag updates.

Step 6.1: Reference DAGs in Policy

1. Navigate to **Policies > Security** (in the appropriate Device Group).
2. Click **Add** to create a new rule, or select an existing rule to modify.
3. In the **Source** or **Destination** field, click **Add** and select the Dynamic Address Group.
4. Configure the rest of the rule: application, service, action, and logging.
5. Click **OK**.

Example rule — allow web traffic from tagged web servers:

FIELD	VALUE
Source	`DAG-WebServers`
Destination	Any
Application	`web-browsing` , `ssl`
Action	Allow

✅ **VERIFICATION**

The security rule appears in the policy rulebase with the DAG name displayed in the Source or Destination column.

Step 6.2: Push to Firewalls

1. Navigate to **Commit > Push to Devices**.
2. Select the Device Groups containing the DAGs and policy rules.
3. Click **Push**.

> ℹ️ **TAGS PUSH AUTOMATICALLY**
>
> Tags are pushed automatically via the plugin — no commit or push is needed for tag updates. However, DAG definitions and security policy rules must be pushed via **Commit > Push to Devices**.

> ℹ️ **DEVICE GROUP RECURSION**
>
> By default, tags are only pushed to Device Groups directly specified in the Notify Group. To propagate tags to child Device Groups, run:

```
debug dau settings device-group recursive yes
```

> ✅ **VERIFICATION**
>
> The push completes with status `Completed` for all target Device Groups. No errors appear in the push results.

Phase 7: Verification

Confirm the full pipeline is working — from AWS metadata retrieval through to security policy enforcement on the firewall dataplane.

Step 7.1: Verify on Panorama

Run these commands on the Panorama CLI to confirm the plugin is polling and processing tags.

Check monitoring definition status:

```
show plugins aws vm-mon-status
```

Expected output shows Mon-Def Name, VPC, Status (`Success`), and Last Updated Time.

Check tag retrieval counters:

```
show plugins aws counters
```

View DAGs on Panorama:

```
set system setting target device-group <DG-NAME>
show object dynamic-address-group all
```

✅ **VERIFICATION**

`show plugins aws vm-mon-status` shows `Success` for all Monitoring Definitions with a recent Last Updated timestamp. `show plugins aws counters` shows non-zero retrieval counts.

Step 7.2: Verify on Firewall

Run these commands on a managed firewall's CLI to confirm tags arrived and DAGs are populated.

View all registered IPs and tags:

```
show object registered-ip all
```

View DAG membership:

```
show object dynamic-address-group all
```

Expected output shows the DAG name, filter (tag match criteria), and member IPs.

Check security policy is using DAGs:

```
show running security-policy
```

View IP-tag registration and unregistration events:

```
show log iptag direction equal backward
```

✅ **VERIFICATION**

`show object registered-ip all` displays IPs with AWS tags. `show object dynamic-address-group all` shows DAGs with member IPs matching the filter. `show running security-policy` shows the DAG-based rules with resolved members.

Step 7.3: Verify Tag Updates (End-to-End)

Confirm the full pipeline updates dynamically when EC2 instances change.

1. In the AWS Console, tag a new EC2 instance (or modify an existing tag) with a key-value pair matching a DAG filter.
2. Wait one monitoring interval (default 60 seconds).
3. On Panorama CLI, check the timestamp updated:

```
show plugins aws vm-mon-status
```

1. On the firewall CLI, verify the new IP and tag appear:

```
show object registered-ip all
```

1. Verify DAG membership updated:

```
show object dynamic-address-group all
```

> ✅ **VERIFICATION**
> The new instance IP appears in the registered-ip list with the expected tags, the DAG membership includes the new IP, and security policy allows or denies traffic as expected. The full pipeline — from AWS tag change to firewall policy enforcement — completes within one monitoring interval.

Troubleshooting

Common failure modes and their diagnostic steps. Work through the pipeline stages in order: retrieval, processing, push, registration.

8.1: Monitoring Definition Status Shows Failure

Common causes:

- Invalid IAM credentials (expired or wrong access keys)
- Missing IAM permissions (verify policy matches Step 2.1)
- Incorrect VPC ID (typo or wrong region)
- Panorama cannot reach AWS API endpoints (check outbound connectivity on ports `443`)
- AWS region not set for GovCloud

Check the retrieval log for detailed errors:

```
less plugins-log plugin_aws_ret.log
```

> ✅ **VERIFICATION**
>
> After fixing the root cause and committing, the Monitoring Definition status changes to `Success` and `plugin_aws_ret.log` shows successful retrieval entries.

8.2: Tags Not Appearing on Panorama

Check the retrieval and processing logs:

```
tail follow yes plugins-log plugin_aws_ret.log
```

```
tail follow yes plugins-log plugin_aws_proc.log
```

Common causes:

- EC2 instance is stopped (must be `running` to be included)
- Tag name exceeds 128 characters (silently ignored)
- Tag contains unsupported special characters
- Monitoring Definition is disabled

✅ **VERIFICATION**

After resolving the issue, `plugin_aws_ret.log` shows the instance being retrieved and `plugin_aws_proc.log` shows tags being processed without errors.

8.3: Tags on Panorama But Not on Firewall

Tags are visible on Panorama but not reaching the firewall. Check the push pipeline:

On Panorama:

```
tail follow yes mp-log configd.log
```

```
tail follow yes mp-log ms.log
```

On firewall:

```
tail follow yes mp-log userid.log
```

Common causes:

- Firewall not in a Device Group listed in the Notify Group
- Device Group recursion not enabled (tags only go to directly listed Device Groups)
- 32 tags/IP limit exceeded — the firewall silently drops additional tags
  - Check: `less mp-log useridd.log` for `total number of tags exceeds the limit 32`
- Firewall also receiving tags from VM Information Source (VIS) — combined total exceeds 32
  - Solution: Disable VIS if using the Panorama plugin

✅ **VERIFICATION**

After resolving the issue, `show object registered-ip all` on the firewall shows the expected IPs and tags. If the 32-tag limit was the cause, `useridd.log` no longer shows limit-exceeded warnings.

8.4: DAG Shows No IPs on Firewall

The DAG exists on the firewall but shows no member IPs.

- The DAG must be referenced in an active security policy rule
- If the DAG is not used in any policy, the firewall shows `members: (not in use)`

```
show object dynamic-address-group all
```

ℹ️ **FIREWALL DAG BEHAVIOR**

Unlike Panorama, firewalls only populate DAGs that are actively referenced in security policy rules. This is by design to conserve memory.

✅ **VERIFICATION**

After adding the DAG to a security policy rule and pushing, `show object dynamic-address-group all` shows member IPs under the DAG.

8.5: HA Failover Issues

Panorama HA

- Active Panorama does **not** sync IP-tag mappings to the passive peer
- On failover, the newly active Panorama triggers a full sync from AWS
- If any Monitoring Definition fails to reconnect, a system log is generated:

```
Unable to process accounts after HA switch-over; user-intervention required.
```

- Tags retrieved before failover are retained on firewalls until Panorama reconnects and updates them
- Resolution: Log in to newly active Panorama, fix invalid Monitoring Definitions, and commit

Firewall HA

- Panorama pushes tags to both active and passive firewalls
- On firewall failover, the newly active firewall already has current IP-tag mappings

✅ **VERIFICATION**

After Panorama HA failover, `show plugins aws vm-mon-status` on the newly active peer shows `Success` for all Monitoring Definitions. After firewall failover, `show object registered-ip all` on the newly active firewall shows current tags.

8.6: Enable Debug Logging

Enable debug logging for deep troubleshooting. Always disable after investigation is complete.

Enable debug on Panorama:

```
request plugins debug level high plugin-name aws
debug management-server on debug
debug management-server set all
```

Key log files on Panorama:

LOG FILE	CONTENTS
`plugin_aws_ret.log`	IP address and tag retrieval from AWS
`plugin_aws_proc.log`	Tag processing and push to configd
`plugin_aws.log`	Plugin configuration, daemons, tag-to-IP mappings (debug mode)
`configd.log`	Revision diffs, push to management server
`ms.log`	XML-API calls to firewalls

Enable debug on firewall:

```
debug user-id on debug
debug user-id set all
```

Key log files on firewall:

LOG FILE	CONTENTS
`userid.log`	IP-to-tag registration events
`devsrv.log`	DAG incremental updates pushed to dataplane

⚠️ **IMPORTANT: DISABLE DEBUG LOGGING AFTER TROUBLESHOOTING**

Debug logging generates significant output and can impact performance. Always disable when investigation is complete:

On Panorama:

```
debug management-server on info
request plugins debug level off plugin-name aws
```

On firewall:

```
debug user-id on info
```

✅ **VERIFICATION**

After disabling, confirm debug is off: `debug management-server show` on Panorama should show `info` level. The plugin debug level reverts to `off`.

CLI Quick Reference

Commonly used commands for managing and troubleshooting the AWS plugin, organized by scope.

9.1: Panorama Plugin Commands

COMMAND	DESCRIPTION
`show plugins aws vm-mon-status`	Monitoring Definition status and last update time
`show plugins aws counters`	Plugin counter statistics
`request plugins aws sync`	Force manual sync with AWS (full re-pull)
`request plugins debug level high plugin-name aws`	Enable debug logging
`request plugins debug level off plugin-name aws`	Disable debug logging
`request plugins dau plugin-name aws unblock-device-push yes`	Unblock tag push when other plugins are stalled
`request plugins aws set-aws-region region <region>`	Set AWS region (GovCloud)

✅ **VERIFICATION**

Run `show plugins aws vm-mon-status` to confirm the plugin is active and `show plugins aws counters` to check retrieval and push statistics.

9.2: DAU (Dynamic Address Update) Commands

COMMAND	DESCRIPTION
`debug dau show job-queue-info`	Show enqueue/dequeue job history with timestamps
`debug dau show tags dgs-and-revs table-name dfas`	Show revision numbers per device group
`debug dau show tags store-tags-ip-rev revision <rev> device-group <dg>`	Decompress revision for inspection
`debug dau settings device-group recursive yes`	Propagate tags to child device groups
`debug dau clear database device-group <dg>`	Clear tag database for a device group
`debug dau show stats`	Show DAU statistics

✅ **VERIFICATION**

Run `debug dau show stats` to see the current DAU state. Non-zero enqueue and dequeue counts indicate tags are flowing through the pipeline.

9.3: Firewall Verification Commands

COMMAND	DESCRIPTION
`show object registered-ip all`	All registered IPs and their tags
`show object dynamic-address-group all`	All DAGs with member IPs
`show running security-policy`	Active security policy with resolved DAG members
`show log iptag ip in <ip>`	Registration/unregistration events for a specific IP
`show log iptag tag_name equal <tag>`	Registration events for a specific tag

✅ **VERIFICATION**

Run `show object registered-ip all` on any managed firewall to confirm IP-to-tag mappings are present and current.

Monitored Attributes

The plugin creates tags from two sources: predefined attributes (always available for every EC2 instance) and user-defined tags (from AWS EC2 instance tags configured by administrators).

10.1: Predefined Tags

These tags are automatically generated from EC2 instance metadata. No AWS-side tagging is required.

TAG NAME	DESCRIPTION	FORMAT EXAMPLE
`aws.ec2.ami-id`	AMI ID	`aws.ec2.ami-id.ami-0abc123`
`aws.ec2.availability-zone`	Availability Zone	`aws.ec2.availability-zone.us-east-1a`
`aws.ec2.account-number.ownerId`	AWS Account ID	`aws.ec2.account-number.ownerId.123456789012`
`aws.ec2.iam-instance-profile`	IAM Instance Profile	`aws.ec2.iam-instance-profile.MyProfile`
`aws.ec2.instance-id`	EC2 Instance ID	`aws.ec2.instance-id.i-0abc123`
`aws.ec2.instance-state`	Instance State	`aws.ec2.instance-state.running`
`aws.ec2.instance-type`	Instance Type	`aws.ec2.instance-type.t3.micro`
`aws.ec2.placement.group-name`	Placement Group	`aws.ec2.placement.group-name.MyGroup`
`aws.ec2.sg-id`	Security Group ID	`aws.ec2.sg-id.sg-0abc123`
`aws.ec2.sg-name`	Security Group Name	`aws.ec2.sg-name.my-sg`
`aws.ec2.sshkey-name`	SSH Key Name	`aws.ec2.sshkey-name.my-key`
`aws.ec2.subnet-id`	Subnet ID	`aws.ec2.subnet-id.subnet-0abc123`
`aws.ec2.vpc-id`	VPC ID	`aws.ec2.vpc-id.vpc-0abc123`
`aws.lb.lb-name`	Load Balancer Name	`aws.lb.lb-name.my-alb`

TAG NAME	DESCRIPTION	FORMAT EXAMPLE
`aws.endpoint-id`	VPC Endpoint ID	`aws.endpoint-id.vpce-0abc123`
`aws.ec2.architecture`	CPU Architecture	`aws.ec2.architecture.x86_64`

✅ **VERIFICATION**

Run `show object registered-ip all` on a managed firewall. Each registered IP should have predefined tags matching the format examples above.

10.2: User-Defined Tags

User-defined tags map directly to AWS EC2 instance tags (key-value pairs configured on the instance in the AWS Console or via API).

Format: `aws.ec2.tag.<key>.<value>`

Example: If an EC2 instance has tag Key=`Environment`, Value=`Production`, the plugin creates:

```
aws.ec2.tag.Environment.Production
```

Limits:

- Maximum 18 user-defined tags per IP (beyond 18, first-come-first-served from the AWS API response)
- Combined with 14 predefined tags = 32 total tags per IP (firewall limit enforced by the userid process)
- Tag name + value must not exceed 128 characters

> ⚠️ **IMPORTANT: 32 TAGS/IP LIMIT**
>
> The 32 tags-per-IP limit is enforced by the firewall's userid process. If an IP has more than 32 tags (e.g., from both the AWS plugin and VM Information Source), the 33rd tag is silently dropped. Disable VM Information Source if using the Panorama plugin to avoid hitting this limit.

> ✅ **VERIFICATION**
>
> Run `show object registered-ip all` on a managed firewall. User-defined tags appear in the format `aws.ec2.tag.<key>.<value>` alongside the predefined tags for each IP.